



Web Development Technologies

Tutorial and Lab Sheet 6

Login & Logout, Sessions and Miscellaneous Concepts

Exercise 1 Complete *mini* web app with login and logout

This is an extension of exercise 1 from tutorial sheet 5, in this example we will add the following features:

- Login and logout with credentials stored in the database
- Lazy Loading
- Session handling in ASP.NET Core
- Customised Routing and
- Generating URLs in the view

We have looked at all these concepts and now is the time to tie them together. This will prove to be a handy exercise for Assignment 2.

Please download the code archive within the Week 6 tutorial material on Canvas. Run the project before analysing the nuts and bolts.

General notes regarding the project:

Login details

LoginID (Customer Name)	Password (Raw)
12345678 (Matthew Bolger)	abc123
38074569 (Rodney Cocker)	ilovermit2020
17963428 (Shekhar Kalra)	youWill_n0tGuess-This!

Added more Controllers

There are 3 controllers:

- HomeController
- LoginController (simple custom routing is used here)
- CustomerController (login required to get into this controller)

Login and Logout code

The salting + hashing algorithm has been used i.e., PBKDF2 which is the same encoding method Identity uses to store passwords. An interesting aspect of PBKDF2 is it is a derived length hash, so you can produce a hash of any desired length, i.e., the hash is not fixed size like in MD5, SHA256, SHA512, etc... this makes it much harder to analysis patterns in the hashes or preparing reverse lookup rainbow tables in advance since you do not know what the length is.

The page protection is implemented with a simple implementation of `IAuthorizationFilter` on an Attribute found in Filters → `AuthorizeCustomerAttribute.cs`:

```
[AuthorizeCustomer]
public class CustomerController : Controller
```

Use Lazy Loading

Question: Why is lazy loading useful?

Found in `CustomerController` → `Index`:

```
// Lazy loading.
// The Customer.Accounts property will be lazy loaded upon demand.
var customer = await _context.Customers.FindAsync(CustomerID);
```

To enable lazy loading, you need to do the following:

- Install the **Microsoft.EntityFrameworkCore.Proxies** NuGet package
- Enable lazy loading in `Startup.cs`:

```
services.AddDbContext<McbaContext>(options =>
{
    options.UseSqlServer(Configuration.GetConnectionString(nameof(McbaContext)));

    // Enable lazy loading.
    options.UseLazyLoadingProxies();
});
```

- Make the navigation properties in the model that you want lazy loading supported **virtual**:

```
public virtual List<Account> Accounts { get; set; }
```

NOTE: You can still use eager loading alongside lazy loading after making these changes.

Use Sessions

The simple in-memory session provider is used (which means every time the webserver is re-built / restarted all sessions are lost).

To enable sessions, add the following to Startup → ConfigureServices:

```
// Store session into Web-Server memory.
services.AddDistributedMemoryCache();
services.AddSession(options =>
{
    // Make the session cookie essential.
    options.Cookie.IsEssential = true;
});
```

The IsEssential flag indicates that session functionality is required for the application to function correctly. Then in a controller you can use HttpContext property to access Session. In a view you can use the Context property.

Additionally, there are some ISession extension methods available when you include the **Microsoft.AspNetCore.Http** namespace:

```
namespace Microsoft.AspNetCore.Http
{
    public static class SessionExtensions
    {
        public static byte[] Get(this ISession session, string key);
        public static int? GetInt32(this ISession session, string key);
        public static string GetString(this ISession session, string key);
        public static void SetInt32(this ISession session, string key, int value);
        public static void SetString(this ISession session, string key, string value);
    }
}
```

Storing an int is simple. If you want to store something “complex” it is recommended you store it as a string and do use serialisation / deserialisation, e.g., JSON.

```
// Login customer.
HttpContext.Session.SetInt32(nameof(Customer.CustomerID), login.CustomerID);
HttpContext.Session.SetString(nameof(Customer.Name), login.Customer.Name);
```

Logout involves simply clearing the session:

```
// Logout customer.
HttpContext.Session.Clear();
```

Customised Routing

Custom routing is used within the LoginController. The Route(...) attribute is used to implement a custom route changing the controller and its actions routes. You can route variables in the URL using this technique as well, although there are not any variables within the custom route implemented here.

```
[Route("/Mcba/SecureLogin")]  
public class LoginController : Controller  
  
[Route("LogoutNow")]  
public IActionResult Logout()
```

Thus, instead of the URL being:

/Login/Login
/Login/Logout

They will now be:

/Mcba/SecureLogin (this will result in LoginCotroller.Login method being called)
/Mcba/SecureLogin/LogoutNow (this will result in LoginController.Logout method being called)

Examples of URL generation in View

Examples of generating URLs in the view demonstrate using the newer Tag-Helpers style and the older HTML-Helpers style:

```
<p>
  <a asp-controller="Login" asp-action="Login">Login link using Tag Helper</a>
</p>
<p>
  @Html.ActionLink("Login link using Html Helper", "Login", "Login")
</p>
```

Code is also present to change the navigation bar based on a user being logged in:

```
<div class="navbar-collapse collapse d-sm-inline-flex flex-sm-row-reverse">
  <ul class="navbar-nav">
    @{
      var isLoggedIn = Context.Session.GetInt32(nameof(Customer.CustomerID)).HasValue;
    }
    @if(isLoggedIn) {
      <li class="nav-item">
        <span class="nav-link text-dark">
          Welcome, @Context.Session.GetString(nameof(Customer.Name))
        </span>
      </li>
      <li class="nav-item">
        <a class="nav-link text-dark" asp-controller="Login" asp-action="Logout">Logout</a>
      </li>
    } else {
      <li class="nav-item">
        <a class="nav-link text-dark" asp-controller="Login" asp-action="Login">Login</a>
      </li>
    }
  </ul>
  <ul class="navbar-nav flex-grow-1">
    <li class="nav-item">
      <a class="nav-link text-dark" asp-controller="Home" asp-action="Privacy">Privacy</a>
    </li>
    @if(isLoggedIn) {
      <li class="nav-item">
        <a class="nav-link text-dark" asp-controller="Customer" asp-action="Index">My Accounts</a>
      </li>
    }
  </ul>
</div>
```

Exercise 2 Assignment Questions

Please spend this time collating and asking assignment related questions.